Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ

з лабораторної роботи № 5
з дисципліни «Основи програмування»
«Умовні оператори»

Виконав:
студент групи ФЕП-13
Марцінко Н. Р.

Перевірив:
асистент Васюта В. М.

Львів – 2026

Мета роботи

Вивчити поняття і застосування умовних операторів.

1. Теоретичні відомості

Оператор if / if–else виконує блок залежно від умови. Конструкція if–else–if дозволяє перевірити кілька умов послідовно. Тернарний оператор умова ? вираз1 : вираз2 компактно повертає один із двох результатів. switch порівнює вираз з константами case і виконує відповідну гілку; break зупиняє провал у наступні case.

2. Завдання 1. Порівняння двох чисел оператором «?»

Програма зчитує a і b та через тернарний оператор виводить «a > b» або «a < b».

2.1. Текст програми (task1.c)

```
Martsinko > Lab5 > c task1.c
1  #include <stdio.h>
2
3  int main () {
4      int a, b;
5      printf("Введіть a і b: ");
6      scanf("%d %d", &a, &b);
7      a > b ? printf("%d > %d", a, b) : printf("%d < %d", a, b);
8      return 0;
9  }
```

Рис. 1. Код task1.c — тернарний оператор

2.2. Результат виконання

```
└>> gcc -Wall -Wextra task1.c -o task1
└>> ./task1
Введіть a і b: 3 55
3 < 55
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└>> ./task1
Введіть a і b: 23 44
23 < 44
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└>> ./task1
Введіть a і b: 4 1
4 > 1
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└>>
```

Рис. 2. Вивід task1: 3 < 55, 23 < 44, 4 > 1

Для пар (3, 55), (23, 44) і (4, 1) умова a > b коректно обирає потрібну гілку printf.

3. Завдання 2. Тип трикутника (if / if–else–if)

Вводяться три сторони. Спочатку перевірка a, b, c > 0 і нерівність трикутника. Далі — рівносторонній / рівнобедрений / різносторонній, потім прямо- / гостро- / тупокутний за теоремою Піфагора та порівнянням квадратів сторін.

3.1. Текст програми (task2.c)

Martsinko > Lab5 > task2.c

```c
#include <stdio.h>

int main () {
    double a, b, c;
    printf("Введіть три сторони трикутника: ");
    scanf("%lf %lf %lf", &a, &b, &c);

    if (a <= 0 || b <= 0 || c <= 0) {
        printf("Помилка: довжини сторін повинні бути більшими за нуль!\n");
        return 1;
    }
    if (a + b > c && a + c > b && b + c > a ) {
        printf("Трикутник існує\n");
    }
    else {
        printf("Трикутник не існує\n");
        return 1;
    }

    if (a == b && b == c) {
        printf("Трикутник є рівностороннім\n");
    }
    else if (a == b || a == c || b == c) {
        printf("Трикутник є рівнобедреним\n");
    }
    else {
        printf("Трикутник є різностороннім\n");
    }

    if (a * a + b * b == c * c || a * a + c * c == b * b || b * b + c * c == a * a) {
        printf("Трикутник є прямокутним\n");
    }
    else if (a * a + b * b > c * c && a * a + c * c > b * b && b * b + c * c > a * a) {
        printf("Трикутник є гострокутним\n");
    }
    else {
        printf("Трикутник є тупокутним\n");
    }

    return 0;
}
```

Рис. 3. Код task2.c

3.2. Результат виконання

```
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>> ./task2
Введіть три сторони трикутника: 1 4 5
Трикутник не існує
Трикутник є різностороннім
Трикутник є тупокутним
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>> gcc -Wall -Wextra task2.c -o task2
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>> ./task2
Введіть три сторони трикутника: 3 64 32
Трикутник не існує
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>> ./task2
Введіть три сторони трикутника: 3 4 5
Трикутник існує
Трикутник є різностороннім
Трикутник є прямокутним
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>> ./task2
Введіть три сторони трикутника: 6 2 9
Трикутник не існує
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>> ./task2
Введіть три сторони трикутника: 12 14 15
Трикутник існує
Трикутник є різностороннім
Трикутник є гострокутним
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>>
```

Рис. 4. Вивід task2: неіснуючі, 3-4-5 (прямокутний), 12-14-15 (гострокутний)

Приклади: 1 4 5 і 3 64 32 — трикутник не існує; 3 4 5 — різносторонній прямокутний; 12 14 15 — різносторонній гострокутний. Перевірка нерівності на початку відсікає некоректні дані.

4. Завдання 3. Тризначне число прописом (switch)

Число розбивається на сотні, десятки й одиниці (/ і %). switch для сотень; для 10–19 — окремий switch за повним остатком; інакше switch для десятків і для одиниць.

4.1. Текст програми (task3.c)

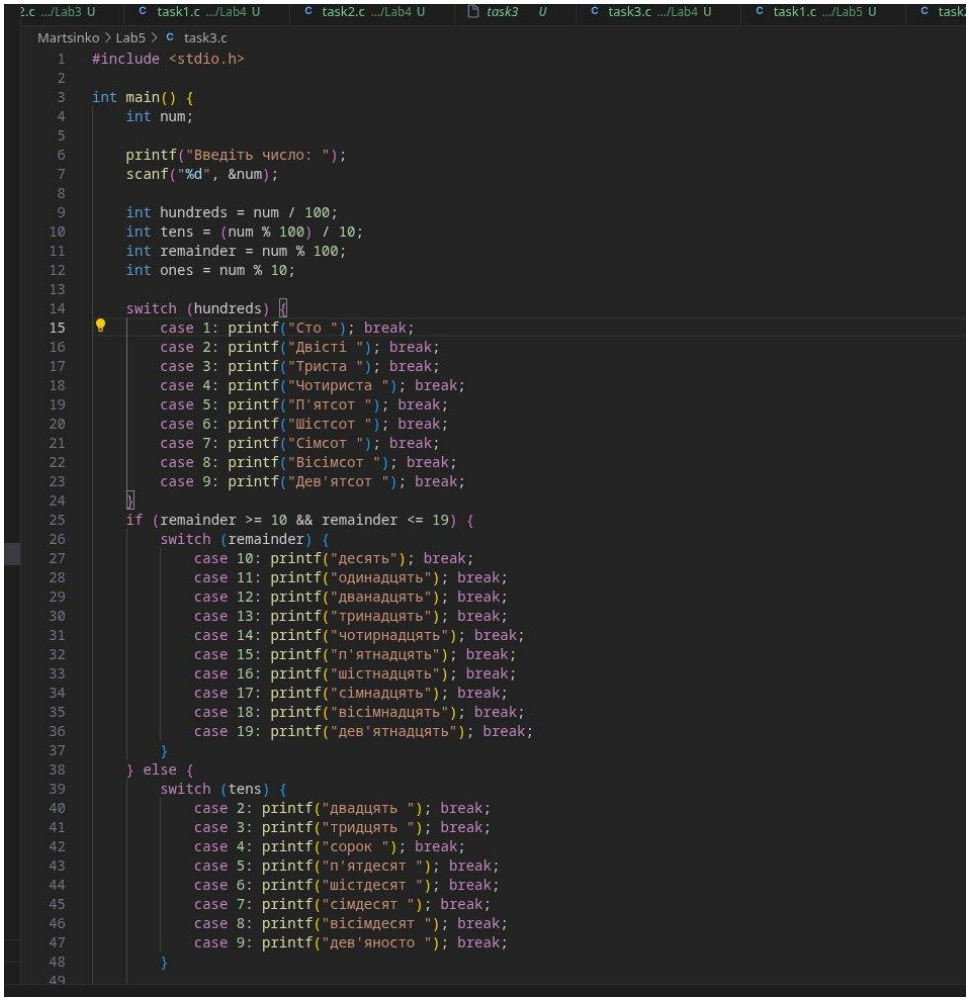
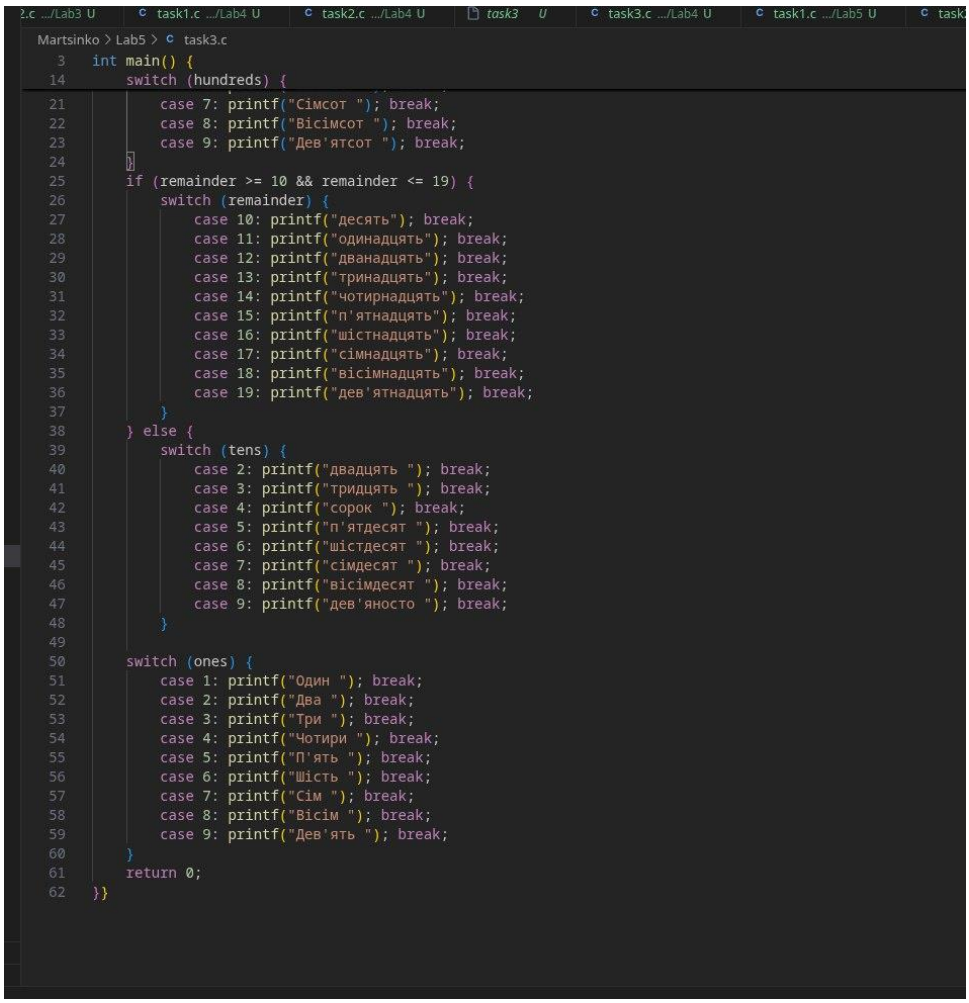


Рис. 5. Код task3.c (частина 1 — сотні та 10–19)



```c
Martsinko > Lab5 > task3.c
int main() {
    switch (hundreds) {
        case 7: printf("Сімсот "); break;
        case 8: printf("Вісімсот "); break;
        case 9: printf("Дев'ятсот "); break;
    }
    if (remainder >= 10 && remainder <= 19) {
        switch (remainder) {
            case 10: printf("десять"); break;
            case 11: printf("одинадцять"); break;
            case 12: printf("дванадцять"); break;
            case 13: printf("тринадцять"); break;
            case 14: printf("чотирнадцять"); break;
            case 15: printf("п'ятнадцять"); break;
            case 16: printf("шістнадцять"); break;
            case 17: printf("сімнадцять"); break;
            case 18: printf("вісімнадцять"); break;
            case 19: printf("дев'ятнадцять"); break;
        }
    } else {
        switch (tens) {
            case 2: printf("двадцять "); break;
            case 3: printf("тридцять "); break;
            case 4: printf("сорок "); break;
            case 5: printf("п'ятдесят "); break;
            case 6: printf("шістдесят "); break;
            case 7: printf("сімдесят "); break;
            case 8: printf("вісімдесят "); break;
            case 9: printf("дев'яносто "); break;
        }

    switch (ones) {
        case 1: printf("Один "); break;
        case 2: printf("Два "); break;
        case 3: printf("Три "); break;
        case 4: printf("Чотири "); break;
        case 5: printf("П'ять "); break;
        case 6: printf("Шість "); break;
        case 7: printf("Сім "); break;
        case 8: printf("Вісім "); break;
        case 9: printf("Дев'ять "); break;
    }
    return 0;
}}
```

Рис. 6. Код task3.c (частина 2 — десятки та одиниці)

4.2. Результат виконання



```
arch in …/2026_OP_FEP_13
└─>> cd /home/arch/projects/2026_OP_FEP_13/Martsinko/Lab5
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>> gcc -Wall -Wextra task3.c -o task3
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>> ./task3
Введіть число: 315
Триста п'ятнадцять
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>> ./task3
Введіть число: 535
П'ятсот тридцять П'ять
arch in …/2026_OP_FEP_13/Martsinko/Lab5
└─>>
```

Рис. 7. Вивід task3: 315 → «Триста п’ятнадцять», 535 → «П’ятсот тридцять п’ять»

Діапазон 10–19 обробляється окремим switch(remainder), щоб уникнути помилок на кшталт «десять п’ять» замість «п’ятнадцять».

Висновки

Під час виконання цієї лабораторної роботи все пройшло гладко та без ускладнень. Усі завдання вдалося реалізувати за допомогою простих умовних конструкцій.

У задачі з трикутником перевірка нерівності на початку дозволила одразу відсікти некоректні дані, після чого тип сторін та кутів легко визначився через звичайні if-else. Використання тернарних операторів також виявилося цілком зручним для швидкого та компактного виводу результатів, жодних проблем із ними не виникло.

У третьому завданні розбір числа на цифри за допомогою операцій / та % пройшов стандартно, а конструкція switch добре підійшла для виводу слів. Єдиний момент, який вимагав окремої уваги — це діапазон чисел від 10 до 19, але він легко закрився додатковим блоком if.